

Cómo crear una instancia de base de datos en la nube

Una base de datos en la nube es un servicio que se proporciona a través de una plataforma de computación en la nube. Ofrece las funcionalidades tradicionales de una base de datos, pero con la flexibilidad y escalabilidad propias de la nube.

Ventajas de las bases de datos en la nube:

- **Facilidad de uso:** Los usuarios pueden acceder desde casi cualquier lugar a través de APIs, interfaces web del proveedor o mediante sus propias aplicaciones, ya sea en la nube o de forma remota.
- **Escalabilidad:** Las bases de datos en la nube permiten aumentar o reducir dinámicamente su capacidad de almacenamiento y procesamiento según la demanda, lo que significa que las organizaciones solo pagan por lo que realmente usan.
- **Recuperación ante desastres:** En caso de eventos como fallos de hardware, cortes de energía o desastres naturales, los datos se mantienen seguros mediante copias de seguridad distribuidas geográficamente en servidores remotos.

Ejemplos de bases de datos relacionales en la nube:

Entre los servicios más conocidos se encuentran IBM Db2 on Cloud, Bases de datos para PostgreSQL en IBM Cloud, el servicio de base de datos en la nube de Oracle, SQL Database de Microsoft Azure y Amazon Relational Database Services (RDS). Estos servicios pueden ejecutarse como máquinas virtuales gestionadas por el usuario o como servicios completamente administrados por el proveedor. Dependiendo del plan, pueden ser de un solo inquilino (dedicado) o de múltiples inquilinos (compartido).

¿Cómo ejecutar una base de datos en la nube?

Para ello, es necesario **aprovisionar una instancia del servicio de base de datos** en la plataforma de nube elegida. Este tipo de solución, conocido como **Database as a Service (DBaaS)**, brinda acceso a los recursos sin tener que preocuparse por la configuración del hardware, la instalación del software o la administración del sistema. Los datos se almacenan en tablas relacionales y pueden ser gestionados desde una interfaz web o mediante APIs, lo cual facilita su integración con aplicaciones.

Una vez conectada la aplicación, esta puede enviar consultas SQL a la instancia, la cual procesará dichas instrucciones y devolverá los resultados solicitados.

Ejemplo: Crear una instancia en IBM Db2 on Cloud

IBM Db2 on Cloud es un servicio de base de datos SQL administrado, que se utiliza igual que cualquier otro sistema de bases de datos, pero sin la carga de adquirir, instalar o mantener el hardware y software necesarios.

Para configurar una instancia:

1. Ve al catálogo de IBM Cloud y selecciona el servicio **Db2**.
2. Hay varias opciones, como Db2 Hosted y Db2 Warehouse. Para este ejemplo, selecciona el plan **Db2 Lite**, que es gratuito.
3. Ajusta la configuración si es necesario: asigna un nombre a la instancia, elige la región, la organización y el espacio de trabajo.
4. Haz clic en **Crear**. La instancia aparecerá en el panel de servicios de IBM Cloud, desde donde podrás gestionarla.

Desde este panel puedes acceder a la **consola web**, donde es posible crear y explorar tablas, cargar datos y ejecutar consultas SQL.

Acceso desde tus aplicaciones

Para acceder desde una aplicación, necesitarás **credenciales de servicio**, las cuales puedes generar desde IBM Cloud. Estas credenciales se presentan como un fragmento JSON que contiene:

- Nombre de la base de datos
- Puerto y host (servidor en la nube donde se encuentra la instancia)
- Nombre de usuario y contraseña

Tener en cuenta que el **nombre de usuario** también será el nombre del **esquema** en el que se crearán las tablas por defecto.

Uso de patrones de cadenas y rangos en SQL

El propósito principal de un sistema de gestión de bases de datos no es solo almacenar datos, sino también facilitar su recuperación de manera eficiente.

En su forma más simple, una sentencia SQL para recuperar datos es:

```
SELECT * FROM nombre_de_tabla;
```

También es posible recuperar un **subconjunto de columnas** específicas, como el ID del libro y el título. Además, podemos **restringir** los resultados utilizando la cláusula WHERE. Por ejemplo:

```
SELECT title FROM book WHERE book_id = 'B1';
```

Pero ¿qué ocurre si no sabemos con exactitud qué valor utilizar en la cláusula WHERE?

La cláusula `WHERE` siempre requiere un **predicado**, es decir, una condición que puede evaluarse como verdadera, falsa o desconocida. Si no conocemos con certeza el valor, como el nombre completo del autor, pero recordamos que **comienza con la letra "R"**, podemos usar **patrones de cadenas** para hacer búsquedas aproximadas.

Uso de patrones de cadena con `LIKE`

En bases de datos relacionales, el operador `LIKE` permite buscar valores que coincidan parcialmente con un patrón. Se utiliza junto con el **carácter comodín** `%`, que representa uno o más caracteres faltantes.

- `%` al final del patrón: busca valores que **empiecen** con cierto texto.
- `%` al inicio: busca valores que **terminen** con cierto texto.
- `%` al inicio y al final: busca valores que **contengan** el texto.

Ejemplo:

Si solo recordamos que el nombre del autor empieza con "R", podemos escribir:

```
SELECT first_name FROM author WHERE first_name LIKE 'R%';
```

Esta consulta devolverá todos los autores cuyo primer nombre comience con R, como por ejemplo "Raoul" y "Rav".

Uso de rangos de valores

Supongamos que queremos recuperar los libros cuyo número de páginas está entre 290 y 300. Una forma de hacerlo es:

```
SELECT * FROM book WHERE pages >= 290 AND pages <= 300;
```

Pero SQL nos permite simplificar esta condición utilizando el operador `BETWEEN`, que incluye ambos extremos del rango:

```
SELECT * FROM book WHERE pages BETWEEN 290 AND 300;
```

El conjunto de resultados será el mismo, pero la consulta es más legible y concisa.

Uso del operador `IN` para múltiples valores

Cuando queremos buscar registros que coincidan con **varios valores específicos**, escribir múltiples condiciones con `OR` puede ser tedioso y repetitivo. Por ejemplo:

```
SELECT * FROM author WHERE country = 'Australia' OR country = 'Brazil';
```

En su lugar, se puede usar el operador IN, que permite especificar una **lista de valores** directamente en la cláusula WHERE:

```
SELECT * FROM author WHERE country IN ('Australia', 'Brazil');
```

Esto se vuelve especialmente útil si queremos incluir más países, como **Canadá, India y China**, sin repetir la misma condición:

```
SELECT * FROM author WHERE country IN ('Canada', 'India', 'China');
```

Ordenar conjuntos de resultados en SQL

El propósito principal de un sistema de gestión de bases de datos no es solo almacenar los datos, sino también facilitar su recuperación de manera eficiente.

Es posible consultar solo ciertas columnas, por ejemplo, los títulos de los libros:

```
SELECT title FROM book;
```

Aunque esta consulta funciona correctamente, **el orden de los resultados puede parecer aleatorio**. Para hacer que el conjunto de resultados sea más claro y fácil de leer, podemos ordenarlos alfabéticamente utilizando la cláusula ORDER BY.

Uso de ORDER BY

La cláusula ORDER BY se utiliza para ordenar los resultados de una consulta en función de una columna específica. Por ejemplo, para ordenar los títulos de los libros en orden alfabético:

```
SELECT title FROM book ORDER BY title;
```

De forma predeterminada, SQL ordena los resultados en **orden ascendente (ASC)**. Por eso, los títulos se muestran en orden alfabético desde la A hasta la Z.

Orden descendente con DESC

Si deseamos ordenar los resultados en orden descendente, debemos usar la palabra clave DESC:

```
SELECT title FROM book ORDER BY title DESC;
```

En este caso, los títulos aparecerán desde la Z hasta la A. Cuando varios títulos comienzan con las mismas palabras, la ordenación se realiza comparando carácter por carácter, desde el punto donde difieren.

Ordenar por posición de columna

Otra manera de ordenar los resultados es indicando la **posición** de la columna en lugar de su nombre. Por ejemplo:

```
SELECT title, pages FROM book ORDER BY 2;
```

Aquí, el número **2** indica que los resultados se deben ordenar según la **segunda columna** listada en la instrucción `SELECT`, que en este caso es `pages`. Esta columna representa el número de páginas del libro.

Esta técnica es útil cuando quieres ordenar por columnas complejas o cuando no deseas repetir el nombre exacto de la columna.

Agrupación de conjuntos de resultados en SQL

Basándonos en un modelo simplificado de base de datos de biblioteca, supongamos que tenemos una tabla llamada `author`, la cual incluye una columna llamada `country`, que contiene el código de país (de dos letras) del autor.

Si realizamos la siguiente consulta:

```
SELECT country FROM author ORDER BY country;
```

Obtendremos una lista con los países de origen de todos los autores, ordenada alfabéticamente. En este ejemplo, hay 20 autores, por lo tanto, el conjunto de resultados tiene 20 filas. Sin embargo, varios autores pueden pertenecer al mismo país, lo que genera filas duplicadas.

--Eliminación de duplicados con `DISTINCT`

Para obtener una lista única de países (sin duplicados), usamos la palabra clave `DISTINCT`:

```
SELECT DISTINCT country FROM author ORDER BY country;
```

Esta consulta reduce el conjunto de resultados a solo 6 filas, mostrando únicamente los países diferentes representados por los autores.

--Agrupar y contar con GROUP BY

Ahora bien, si queremos saber cuántos autores hay por cada país, utilizamos la cláusula `GROUP BY` junto con la función de agregación `COUNT`:

```
SELECT country, COUNT(*) FROM author GROUP BY country;
```

Esta consulta agrupa a los autores por país y cuenta cuántos pertenecen a cada uno. El resultado muestra dos columnas: el país y el número de autores de ese país.

--Asignar un nombre a la columna calculada con AS

Para que la columna que contiene el conteo tenga un nombre más claro, podemos asignarle un alias usando `AS`:

```
SELECT country, COUNT(*) AS total_autores FROM author GROUP BY country;
```

Esto hace que el conjunto de resultados sea más fácil de interpretar, mostrando una columna llamada `total_autores`.

--Filtrar resultados agrupados con HAVING

Si queremos mostrar solo los países que tienen más de 4 autores, no podemos usar la cláusula `WHERE` porque esta actúa sobre los datos antes de que se agrupen. En cambio, usamos la cláusula `HAVING`, que se aplica **después** del `GROUP BY`:

```
SELECT country, COUNT(*) AS total_autores
FROM author
GROUP BY country
HAVING COUNT(*) > 4;
```

Esta consulta devuelve únicamente los países donde hay **más de 4 autores**. Por ejemplo, si China e India tienen 6 autores cada uno, solo ellos aparecerán en el resultado.

db2 => select country, count(country) as Count from Author group by country	
Country	Count
AU	1
BR	1
CA	3
CN	6
IN	6
RO	3
6 record(s) selected.	

db2 => select country, count(country) as Count from Author group by country having count(country) > 4	
Country	Count
CN	6
IN	6
6 record(s) selected.	

